

RÉPUBLIQUE DU CAMEROUN

Paix-Travail-Patrie

Université de Yaoundé I

École Nationale Supérieure
Polytechnique de Yaoundé (ENSPY)

BP 8390 Tel/Fax: 222 22 45 47
Yaoundé, Cameroun



REPUBLIC OF CAMEROON

Peace-Work-Fatherland

The University of Yaounde I

National Advanced School of
Engineering of Yaounde (NASEY)

PO BOX 8390 Tel/Fax: 222 22 45 47
Yaounde, Cameroon

ECOLE NATIONALE SUPERIEURE POLYTECHNIQUE DE YAOUNDE (ENSPY)

DEVELOPPEMENT D'UN ASSISTANT MÉDICAL VOCAL INTELLIGENT

Département : Génie Informatique

Filière : Arts Numérique

Spécialité : Art et Intelligence Artificielle

Niveau : 4

Unité d'Enseignement (UE) : Agent Intelligents

Code de l'UE : ANI-IA 4098

Enseignant : Mme Sonfack.

<i>Matricule</i>	<i>Nom</i>
<i>23P734</i>	<i>EDENE LUCIE SAINT PIERRE</i>
<i>21P553</i>	<i>NGUETSA TAMOKA</i>

Année Académique : 2024/2025

Table des matières

Table des matières.....	1
Introduction.....	3
2. Problématique & Cas d'usage.....	3
2.1 Difficultés d'accès aux conseils médicaux	3
2.2 Charge cognitive des professionnels de santé.....	4
2.3 Cas d'usage cible : chatbot pré-diagnostic voix-texte-voix	4
3. Revue de littérature et état de l'art	5
3.1 Chatbots de santé : MedPaLM, Florence, Babylon Health.....	5
3.2 STT : Whisper, alternatives locales, temps réel	5
3.3 LLMs : Falcon, LoRA, BioGPT, ClinicalBERT	6
3.4 TTS : gTTS, ElevenLabs, Coqui.....	6
3.5 Synthèse : positionnement de notre projet	7
4. Conception de la solution.....	7
4.1 Architecture fonctionnelle (bloc schéma)	7
4.2 Choix technologiques : FastAPI, Falcon, LoRA, HTML5	8
4.3 Spécifications vocales et multimodales	8
4.4 Plan de fine-tuning et critères	9
5. Modélisation UML de l'architecture	10
5.1 Diagramme de cas d'utilisation : interaction utilisateur–chatbot	10
5.2 Diagramme de classes : structure des services	10
5.3 Diagramme de séquence : voix → texte → IA → voix	12
5. Implémentation technique.....	13
5.1 Environnement de développement.....	13
5.2 Prétraitement et nettoyage des données médicales	13
5.3 Fine-tuning de Falcon avec LoRA	13
5.4 Exportation du modèle.....	14
5.5 Test local du modèle (script interactif)	14
5.6 Intégration backend (FastAPI).....	14
5.7 Intégration frontend (HTML + JS).....	14
6. Implementation	15
7. Difficultés rencontrées	17

1. Limitations de ressources matérielles (GPU / mémoire)	17
2. Intégration vocale (STT + TTS).....	17
3. Problèmes de CORS et interopérabilité Web/API	18
4. Erreurs HTTP 422 et mauvaise gestion des fichiers	18
5. Interception des requêtes par Internet Download Manager (IDM).....	18
6. Absence de mémoire de conversation.....	18
8. Perspectives d'évolution	19
8.1 Ajout de mémoire de contexte conversationnel	19
8.2 Version mobile Flutter.....	19
8.3 WebSocket temps réel	19
8.4 Multilingue / multivoix	20
Conclusion générale.....	20

Introduction

L'intelligence artificielle (IA) est en pleine mutation, et ses applications dans le domaine de la santé ne cessent de se diversifier. Parmi elles, les agents conversationnels ou *chatbots* connaissent un essor rapide, notamment grâce à leur capacité à améliorer l'accessibilité aux services médicaux, à fluidifier la relation soignant-patient, et à automatiser certaines tâches informatives ou de prédiagnostic.

Cependant, la majorité des assistants médicaux existants reposent principalement sur des interfaces textuelles, limitant leur usage aux personnes alphabétisées, connectées et à l'aise avec la saisie écrite. Dans des contextes multilingues ou auprès de populations faiblement lettrées, cette barrière représente un frein considérable à l'adoption. Face à ce constat, l'intégration de la voix comme vecteur principal d'interaction s'impose comme une solution prometteuse.

Ce projet vise à concevoir un **chatbot vocal médical intelligent**, capable de recevoir une question orale de l'utilisateur, de transcrire cette requête en texte, de générer une réponse médicale à l'aide d'un modèle de langage entraîné (type Falcon RW 1B fine-tuné), puis de restituer cette réponse à l'utilisateur sous forme vocale, en temps réel.

L'architecture repose sur une chaîne complète incluant :

- la **transcription vocale (STT)** avec Whisper ou un équivalent robuste,
- le **générateur de réponse textuelle** fondé sur un grand modèle de langage préentraîné et ajusté aux besoins médicaux,
- la **synthèse vocale (TTS)** pour convertir le texte en voix claire et naturelle,
- une **interface utilisateur Web intuitive**, permettant de tester la plateforme directement depuis un navigateur.

Au-delà de l'aspect technique, ce système a pour ambition d'améliorer l'accessibilité des conseils médicaux automatisés dans des zones sous-médicalisées, et de proposer une expérience conversationnelle fluide, même sans clavier ni interface graphique complexe.

L'ensemble du projet a été conçu selon une architecture modulaire en microservices, facilitant le déploiement, l'évolution indépendante des composants (reconnaissance vocale, génération de texte, synthèse vocale) et l'adaptabilité future à d'autres langues, domaines ou modèles.

2. Problématique & Cas d'usage

2.1 Difficultés d'accès aux conseils médicaux

Dans de nombreuses régions du monde, et particulièrement dans les pays à faibles et moyens revenus, l'accès à une information médicale fiable reste limité. Ce déficit est amplifié par plusieurs facteurs : la pénurie de professionnels de santé, la distance géographique des centres

de soins, les barrières linguistiques, le faible niveau d’alphabétisation, et parfois la méfiance vis-à-vis des structures médicales.

L’émergence d’outils technologiques de santé numérique (e-santé) a permis certaines avancées, mais ces solutions nécessitent bien souvent une interface textuelle, un appareil performant, voire une connectivité constante — des prérequis encore trop contraignants pour de nombreux utilisateurs. De plus, les outils actuels sont rarement multilingues, et encore moins adaptés aux interactions naturelles par la voix dans des contextes bruyants ou à faible bande passante.

Ces contraintes laissent une partie importante de la population sans accès à des conseils médicaux de première intention, pourtant essentiels à la prévention, à l’éducation sanitaire ou à l’orientation précoce vers une structure adaptée.

2.2 Charge cognitive des professionnels de santé

Dans les environnements hospitaliers ou communautaires surchargés, les médecins et personnels infirmiers font face à une charge de travail croissante. Celle-ci inclut non seulement des tâches cliniques, mais aussi de nombreuses sollicitations administratives et informationnelles : répondre aux mêmes questions répétitives, gérer des files d’attente, donner des explications simples à des patients angoissés, etc.

Un assistant conversationnel bien conçu pourrait alléger cette charge en prenant en main une partie de ces interactions à faible valeur ajoutée clinique. Il pourrait aussi servir de préfiltre, orientant les patients vers les bonnes ressources ou alertant le personnel en cas d’urgence détectée.

En intégrant une interface vocale fluide, le système se rapproche d’un échange humain naturel, rendant l’expérience plus intuitive pour les patients et moins dépendante d’un dispositif médical formel.

2.3 Cas d’usage cible : chatbot pré-diagnostic voix-texte-voix

Le projet s’inscrit dans une approche de **chatbot vocal médical à double conversion** (voix → texte → réponse → voix), déployé sous forme d’une interface web accessible sur navigateur.

Le cas d’usage principal est celui d’un **assistant de pré-diagnostic** ou de **conseil médical initial**. L’utilisateur (patient ou proche) peut poser une question à l’oral, comme par exemple :

- « *I have throath pain what should I do ?* »
- « *My child caughts in the night ?* »

Le chatbot traite cette entrée vocale grâce à un module de reconnaissance de la parole, génère une réponse médicale cohérente basée sur un modèle de langage fine-tuné sur des données cliniques, puis lit cette réponse à haute voix à l’utilisateur.

Ce flux simplifie la communication et rend possible une première interaction médicale :

- sans besoin de lire ou écrire,
- sans inscription,

- et potentiellement en situation de mobilité ou d'urgence.

Le système peut aussi être envisagé dans d'autres contextes : accompagnement des personnes âgées, soutien dans les pharmacies communautaires, ou intégration dans des bornes de téléconsultation.

3. Revue de littérature et état de l'art

3.1 Chatbots de santé : MedPaLM, Florence, Babylon Health

L'essor de l'intelligence artificielle dans la santé a vu émerger plusieurs chatbots médicaux, destinés à informer, guider ou assister les patients.

MedPaLM, développé par Google, est l'un des systèmes de question-réponse médicale les plus avancés. Basé sur une version spécialisée de PaLM (Pathways Language Model), il a été entraîné sur des examens médicaux (USMLE, MedQA, etc.) pour offrir des réponses fiables, avec un score de concordance avec des médecins supérieurs à 85 %. Toutefois, sa complexité technique et son infrastructure cloud nécessitent des ressources importantes, peu adaptées aux environnements décentralisés ou à faible connectivité.

Florence, le chatbot de santé de l'OMS, est davantage orienté vers la sensibilisation et la prévention. Disponible en plusieurs langues, il utilise une approche conversationnelle textuelle pour aborder des sujets comme le tabagisme, la vaccination ou l'activité physique. Bien qu'accessible, Florence reste limité aux contenus préprogrammés, sans personnalisation des réponses.

Babylon Health, enfin, propose une solution intégrée avec symptom checker, prise de rendez-vous et téléconsultation. Son modèle repose sur des arbres décisionnels enrichis par l'IA, mais les critiques portent sur son manque de transparence dans le raisonnement clinique, et sa faible adaptation aux langues ou cultures non occidentales.

Ces solutions, bien que novatrices, souffrent souvent d'un manque d'accessibilité vocale, d'une complexité d'intégration, ou d'une dépendance à des serveurs distants. D'où l'intérêt d'un chatbot léger, vocal, et déployable localement.

3.2 STT : Whisper, alternatives locales, temps réel

La **reconnaissance vocale (STT : Speech-to-Text)** constitue une brique essentielle de toute interface vocale. L'un des modèles les plus performants à ce jour est **Whisper** d'OpenAI, un modèle multilingue open source qui excelle dans la transcription de discours dans divers environnements sonores.

Whisper est capable de détecter et transcrire des voix humaines même en présence de bruit de fond modéré. Il supporte plus de 50 langues et peut être exécuté localement sur GPU, une caractéristique majeure pour garantir la confidentialité des données vocales sensibles. Son principal inconvénient réside dans sa latence en mode temps réel, bien qu'optimisable.

Parmi les **alternatives locales** :

- **Vosk** : rapide, léger, avec une bonne compatibilité temps réel, mais moins performant sur les accents ou en contexte bruité.
- **Mozilla DeepSpeech** : anciennement populaire, mais moins maintenu.
- **Whisper.cpp** : version optimisée en C++ de Whisper, adaptée aux environnements embarqués, avec une faible consommation mémoire.

Notre choix s'est porté sur Whisper pour sa robustesse, en mettant en place un traitement préliminaire du signal (Voice Activity Detection + Denoising) pour renforcer ses performances temps réel.

3.3 LLMs : Falcon, LoRA, BioGPT, ClinicalBERT

Le **raisonnement médical automatisé** repose sur des **modèles de langage de grande taille (LLMs)** capables de comprendre et de générer du texte médical pertinent.

- **Falcon** (du TII - Technology Innovation Institute) est un modèle open-source efficace, souvent utilisé comme base pour des fine-tunings personnalisés. Son architecture est adaptée aux environnements avec ressources limitées.
- **LoRA (Low-Rank Adaptation)** est une technique permettant d'adapter un LLM avec peu de paramètres entraînaables, réduisant considérablement les coûts de fine-tuning tout en gardant de bonnes performances.
- **BioGPT** et **ClinicalBERT** sont des modèles pré-entraînés sur des textes biomédicaux (PubMed, MIMIC, etc.). Ils présentent une compréhension supérieure du vocabulaire médical, mais sont majoritairement en anglais, et souvent adaptés à des tâches spécifiques comme la classification ou l'extraction d'information.

Le projet repose sur **Falcon + LoRA**, fine-tuné sur des dialogues médicaux pour maximiser la pertinence et la cohérence des réponses vocales, tout en maintenant un déploiement possible en local.

3.4 TTS : gTTS, ElevenLabs, Coqui

La **synthèse vocale (TTS : Text-to-Speech)** complète le cycle d'interaction voix-texte-voix. Plusieurs outils sont disponibles selon les exigences en termes de qualité, latence, et personnalisation :

- **gTTS (Google Text-to-Speech)** est simple, multilingue, et rapide, mais nécessite une connexion Internet.
- **ElevenLabs** offre une qualité de synthèse proche de la voix humaine, avec des intonations naturelles, mais il s'agit d'un service payant, basé sur le cloud.
- **Coqui TTS** est une solution open source, capable de s'exécuter en local, avec des voix personnalisables et un entraînement de modèles multilingues.

Notre choix initial s'est orienté vers **gTTS** pour la simplicité de test, avec une perspective d'évolution vers **Coqui TTS** pour un déploiement autonome et respectueux de la confidentialité vocale.

3.5 Synthèse : positionnement de notre projet

Contrairement aux grandes solutions centralisées, notre projet propose un **assistant médical vocal léger, multilingue, et accessible en local**, intégrant :

- **Whisper** pour la STT en temps réel avec filtrage de bruit,
- **Falcon + LoRA** pour une compréhension médicale spécialisée,
- **gTTS/Coqui** pour la lecture vocale des réponses,
- Une **interface web simple et responsive**, testable sans infrastructure complexe.

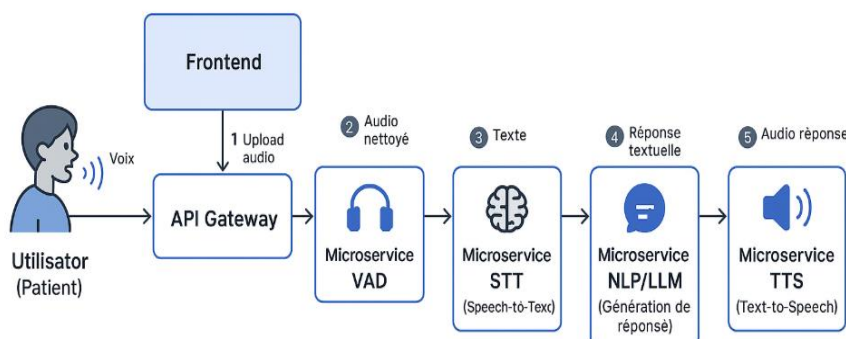
Ce positionnement hybride — entre accessibilité, confidentialité, et spécialisation — en fait une solution viable pour les environnements médicaux de première ligne, les zones rurales, ou les dispositifs embarqués de santé numérique.

4. Conception de la solution

4.1 Architecture fonctionnelle (bloc schéma)

L'architecture de notre assistant médical vocal repose sur une organisation **modulaire en microservices**, orientée **pipeline de traitement audio-texte-audio**, permettant souplesse et scalabilité.

Schéma fonctionnel (vue simplifiée) :



- L'utilisateur interagit via une interface HTML5 avec boutons de commande vocale.
- Chaque composant est encapsulé dans des modules FastAPI pour faciliter la maintenance.
- Le passage entre les étapes est orchestré via des appels HTTP internes (via API Gateway ou routeurs FastAPI).

4.2 Choix technologiques : FastAPI, Falcon, LoRA, HTML5

Composant	Technologie choisie	Motivation
Backend Web API	FastAPI	Rapide, asynchrone, documentation Swagger intégrée
Modèle NLP médical	Falcon-RW-1B + LoRA fine-tuning	Léger, open-source, facilement personnalisable
Transcription vocale (STT)	Whisper (openai/whisper-base) ou whisper.cpp	Robuste, multilingue, fonctionne hors ligne
Synthèse vocale (TTS)	gTTS (temporaire), Coqui TTS (prévu)	Facilité d'intégration vs autonomie future
Frontend interactif	HTML5 + JS (SpeechRecognition, SpeechSynthesis API)	Accessibilité immédiate sans dépendances lourdes
Traitement audio brut	WebRTC, RecordJS, ffmpeg, noise reduction utils	Préparation audio pour transcription fiable

4.3 Spécifications vocales et multimodales

Notre solution repose sur une **interaction bidirectionnelle exclusivement vocale**, tout en permettant un **affichage texte secondaire** pour améliorer l'accessibilité.

Entrée vocale :

- Audio brut capté via micro ou fichier .wav
- Nettoyage : suppression du bruit + Voice Activity Detection
- Langue principale : **Anglais (US)**

Sortie vocale :

- Réponse générée par LLM → convertie en audio via TTS
- Langue de synthèse : **Anglais**
- Optionnel : affichage du texte généré pour transparence

Multimodalité :

- Voix → Texte → Voix
- Voix → Texte seul (si TTS désactivé)
- Texte → Voix (cas de saisie manuelle pour test)

Temps réel :

- Temps de latence maximal toléré : < **3 secondes** par interaction
- Utilisation de modules non bloquants (FastAPI async + TTS local)

4.4 Plan de fine-tuning et critères

Le modèle Falcon utilisé dans ce projet est enrichi par fine-tuning via la méthode **LoRA** (Low-Rank Adaptation), qui permet d'ajuster le modèle avec peu de ressources.

Étapes de fine-tuning :

- 1. Préparation des données :**
 - Dialogues médicaux synthétiques et réels
 - Annotation de dialogues selon le format instruction-réponse
- 2. Nettoyage et tokenisation :**
 - Nettoyage des erreurs, uniformisation des entités médicales
 - Tokenisation via le tokenizer Falcon
- 3. Entraînement avec LoRA :**
 - Sélection des couches à adapter
 - Réduction de la RAM GPU via `load_in_8bit=True`
 - Utilisation de `bitsandbytes` + QLoRA si besoin
- 4. Validation :**
 - Échantillons évalués manuellement
 - Taux de cohérence médicale, hallucination réduite

Critères de performance visés :

- **Pertinence de réponse médicale :** > 80 % (via grille d'évaluation médicale)
- **Latence moyenne par requête :** < 2.5 secondes (hors GPU cloud)
- **Taux de réussite STT + TTS :** > 90 % (par test manuel utilisateur)

5. Modélisation UML de l'architecture

Dans le cadre de ce projet, les diagrammes UML permettent de formaliser les **interactions clés** et la **structure logicielle** du chatbot vocal. Les modèles choisis traduisent fidèlement les composants de l'application, de l'interface utilisateur jusqu'aux microservices backend. Nous avons utilisé la plateforme de génération de diagramme PlantUML ou nous avons passé le code.

5.1 Diagramme de cas d'utilisation : interaction utilisateur–chatbot

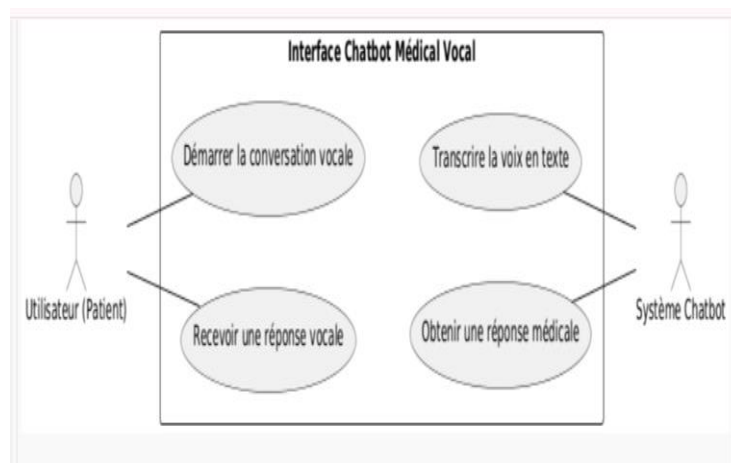
Ce diagramme montre les **interactions principales** entre l'utilisateur et le système.

Acteurs :

- **Utilisateur (Patient)**
- **Système (Chatbot vocal)**

Cas d'utilisation principaux :

- Démarrer la conversation vocale
- Transcrire la voix en texte
- Obtenir une réponse médicale
- Recevoir une réponse vocale
- Lire l'historique des interactions (futur)

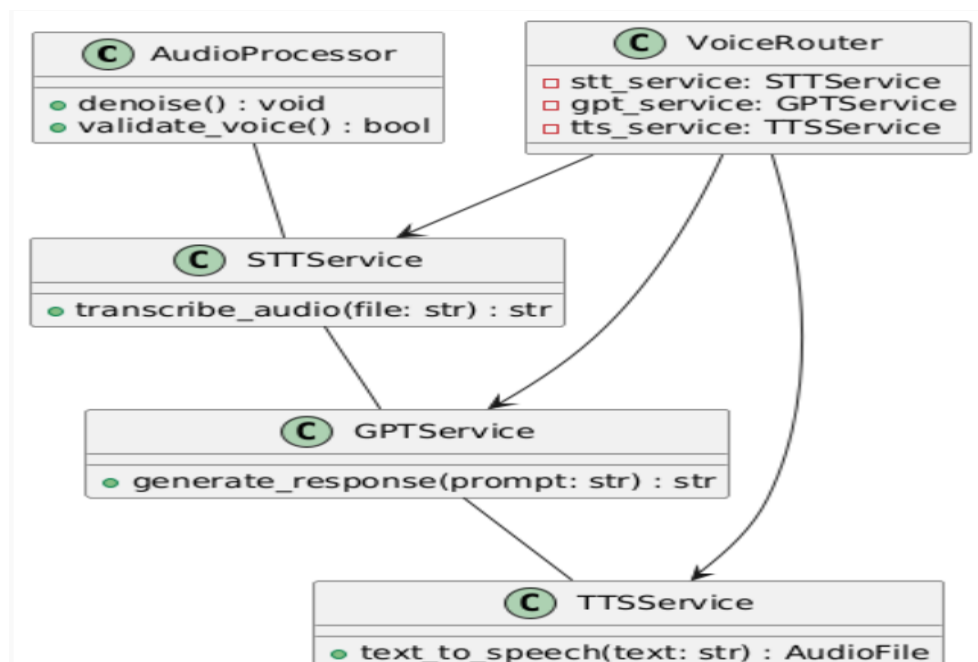


5.2 Diagramme de classes : structure des services

Ce diagramme illustre la structure interne des **composants clés** de l'application backend.

Classes principales :

- AudioProcessor
 - Méthodes : `denoise()`, `validate_voice()`
- STTService
 - Méthodes : `transcribe_audio(file)`
- GPTService
 - Méthodes : `generate_response(prompt)`
- TTSService
 - Méthodes : `text_to_speech(text)`
- VoiceRouter (contrôleur FastAPI)
 - Dépendances : STTService, GPTService, TTSService



5.3 Diagramme de séquence : voix → texte → IA → voix

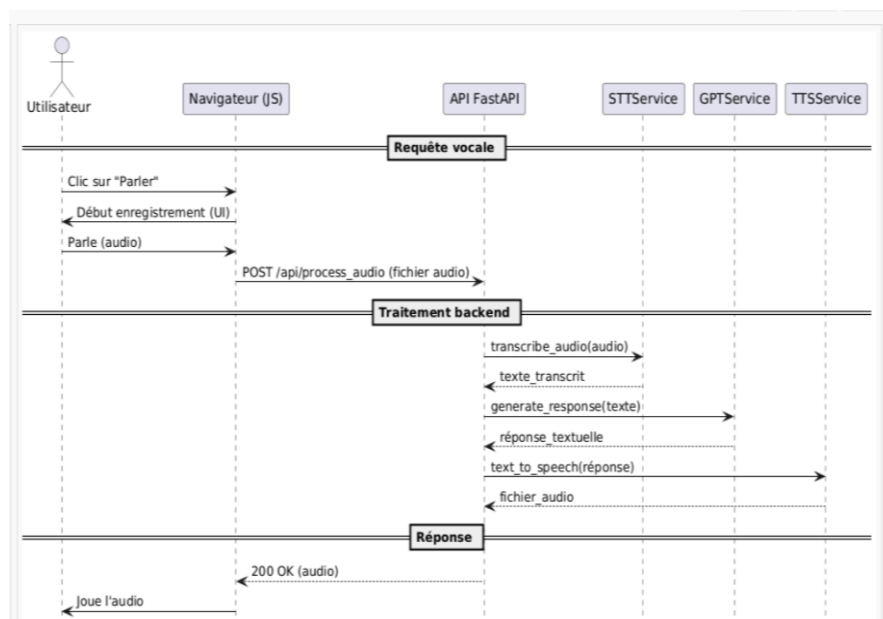
Ce diagramme illustre le **flux d'une requête vocale complète**, du micro jusqu'au retour vocal.

Objets / composants :

- Utilisateur
- Navigateur web (JS)
- API FastAPI
- STTService
- GPService
- TTSService

Étapes typiques :

1. Utilisateur clique sur "Parler"
2. Le navigateur enregistre l'audio et l'envoie au backend
3. STTService transcrit l'audio en texte
4. GPService génère une réponse
5. TTSService convertit la réponse en audio
6. L'API retourne le fichier audio au navigateur



5. Implémentation technique

Cette section décrit l'ensemble du processus de mise en œuvre technique de notre chatbot vocal, de l'environnement de développement jusqu'à l'intégration frontend et la gestion des erreurs.

5.1 Environnement de développement

Le projet repose sur un **ensemble d'outils modernes** pour l'IA vocale et le développement web rapide :

- **Langage** : Python 3.10+
- **Framework backend** : FastAPI
- **Reconnaissance vocale (STT)** : Whisper (via openai-whisper)
- **Modèle de réponse médicale** : Falcon-LoRA (via transformers + peft)
- **Synthèse vocale (TTS)** : navigateur via SpeechSynthesisUtterance, ou gTTS côté serveur
- **Environnement virtuel** : venv
- **Frontend** : HTML5 / JavaScript (natif, sans framework)

5.2 Prétraitement et nettoyage des données médicales

Pour le fine-tuning du modèle, nous utilisons une base de dialogues médicaux :

- **Source** : [ai-medical-chatbot\(EN\)](#)
- **Étapes appliquées** :
 - Suppression des doublons
 - Uniformisation des rôles ("Doctor:", "Patient:")
 - Nettoyage des caractères spéciaux
 - Segmentation en paires question-réponse
 - Filtrage des cas vides ou ambigus
- **Format final** : CSV ou JSONL { "prompt": ..., "response": ... }

5.3 Fine-tuning de Falcon avec LoRA

Nous avons configuré un fine-tuning léger avec l'algorithme **LoRA (Low-Rank Adaptation)**.

- **Modèle de base** : Falcon-7B-instruct
- **Bibliothèques utilisées** :
 - transformers (modèles)

- peft (LoRA)
- accelerate (optimisation)
- **Hyperparamètres :**
 - Epochs : 3–5
 - Batch size : 8–16
 - Learning rate : 2e-5
 - Max length : 512 tokens
- **Sauvegarde :** uniquement les poids LoRA (léger)

5.4 Exportation du modèle

Une fois entraîné, le modèle est exporté au format compatible avec la production :

- **Structure du modèle :** adapter_model.bin + adapter_config.json
- **Stockage local** dans models/falcon-lora-finetuned/

5.5 Test local du modèle (script interactif)

Un script de test (interactive.py) permet de **simuler des dialogues** avec le modèle :

- Lecture en ligne de commande
- Conversion texte–réponse
- Option de sauvegarde de l'historique
- Permet de valider la pertinence des réponses avant déploiement

5.6 Intégration backend (FastAPI)

Le backend contient plusieurs services modulaires :

- /chatbot/voice : reçoit un fichier audio, détecte la voix, transcrit, interroge le modèle, et renvoie un fichier .mp3
- stt_service.py : gestion de Whisper
- gpt_service.py : génération via Falcon-LoRA
- tts_service.py : conversion texte → audio
- Gestion des logs, erreurs et CORS

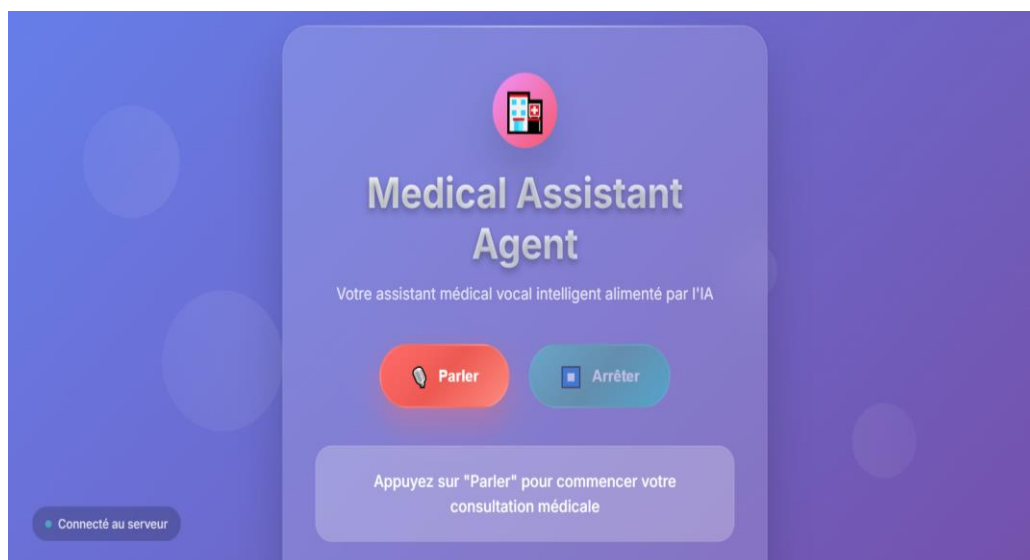
5.7 Intégration frontend (HTML + JS)

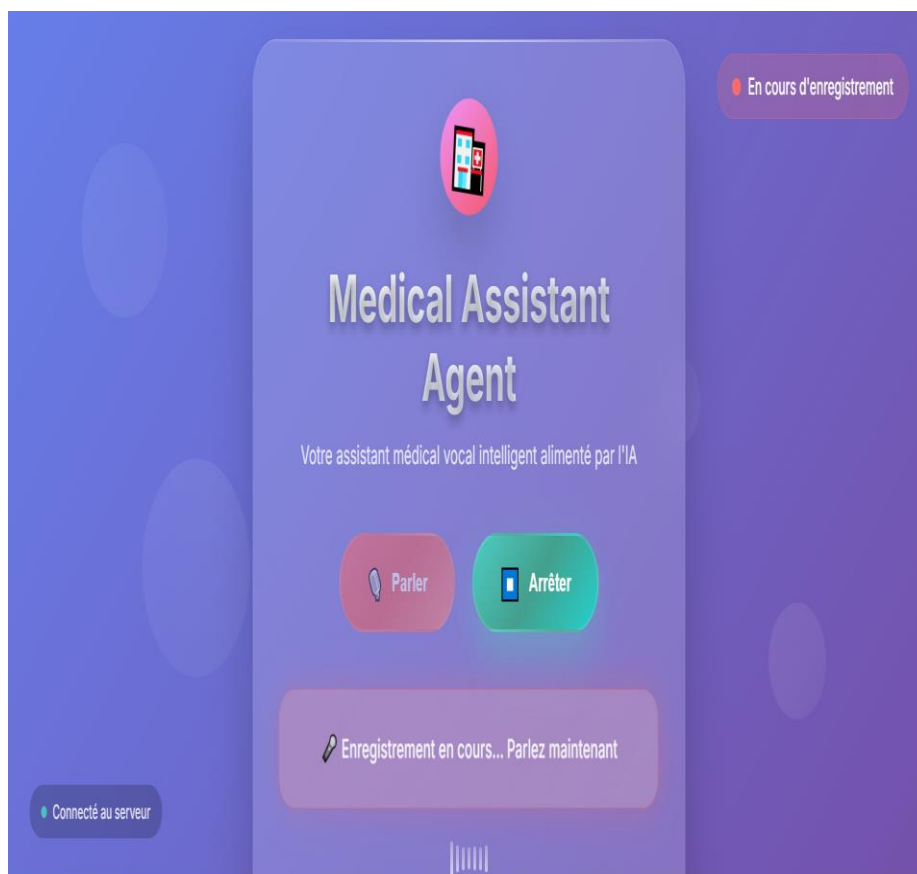
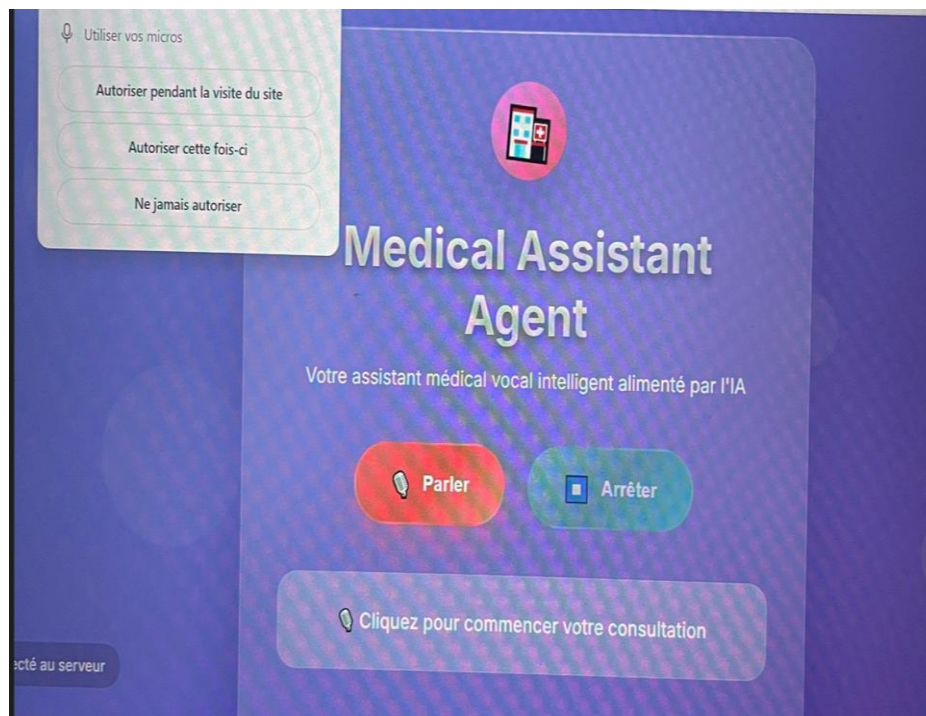
Interface simple basée sur HTML + JavaScript natif :

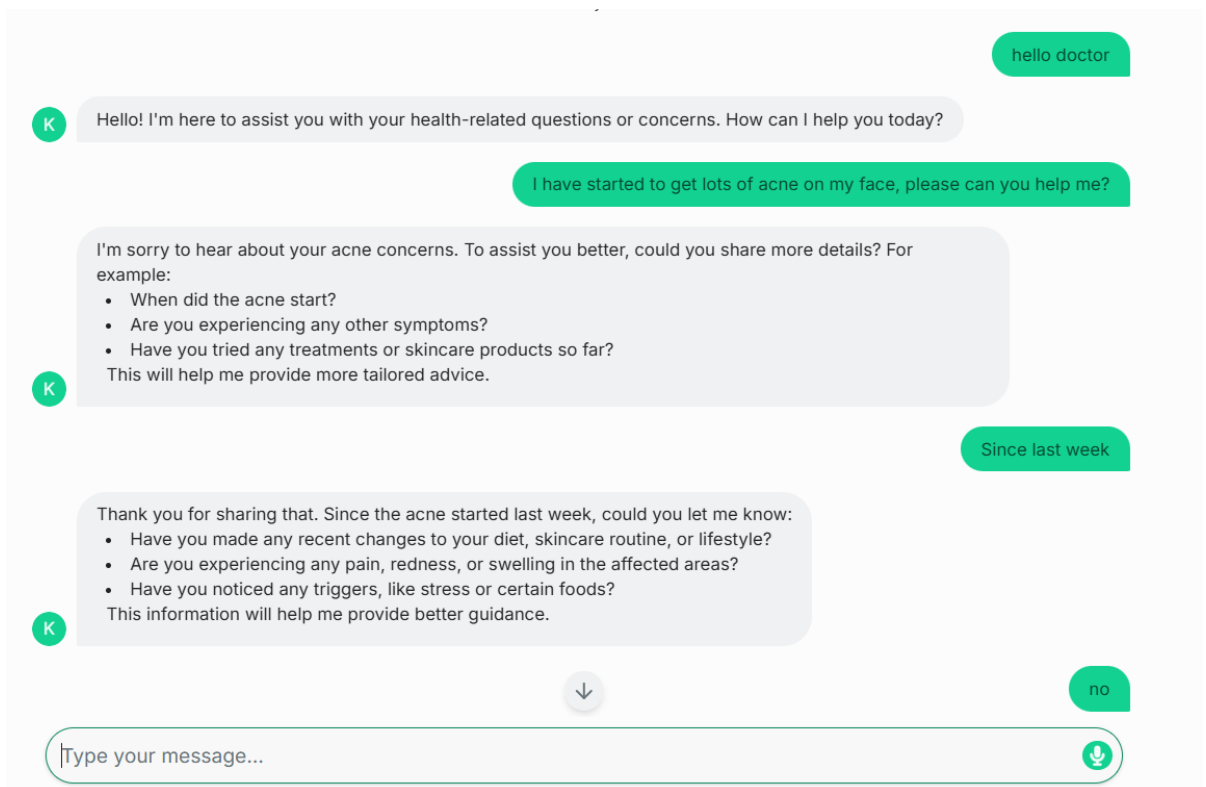
- **STT côté navigateur :**

- SpeechRecognition API (WebKit)
- **TTS côté navigateur :**
 - SpeechSynthesisUtterance
- **Affichage :**
 - Zone de transcription
 - Input texte pour test TTS
 - Boutons “Parler” et “Lire”
- Aucune dépendance externe, fonctionne en local

6.Implementation







7. Difficultés rencontrées

Nous avons encore quelques difficultés avec l'envoi et la réception de la voie à travers le micro de la machine, nous avons néanmoins tester les modules indépendants avec des audio et ça marche. Nous avons passé un audio pré-enregistré et le system génère effectivement une réponse audio. En plus de cette difficulté technique, nous avons eu les difficultés suivantes.

1. Limitations de ressources matérielles (GPU / mémoire)

Le fine-tuning d'un grand modèle de langage comme Falcon, même avec LoRA et quantization 8-bit, reste coûteux en mémoire.

- Les sessions sur Kaggle T4 étaient limitées à quelques heures et 15 Go de VRAM.
- Le modèle a parfois provoqué des erreurs OutOfMemoryError, forçant la réduction de batch size ou du nombre d'échantillons.

Solution adoptée : entraînement sur un sous-échantillon (5k paires), LoRA, load_in_8bit, désactivation de certains callbacks, et sauvegarde incrémentale.

- Le cota d'heure limites sur kaggle pour utiliser le GPU

2. Intégration vocale (STT + TTS)

- Les navigateurs n'offrent pas tous un support stable de la reconnaissance vocale (SpeechRecognition API).

- Le module Whisper local nécessite des prétraitements audio pour fonctionner correctement : silence, bruit de fond, formats.
- TTS côté client (SpeechSynthesisUtterance) varie selon le système (Windows, Linux, Android) → voix incohérentes, pas toujours naturelles.

Solutions :

- Ajout de `denoise_audio()` + VAD pour améliorer la transcription.
- Alternative TTS serveur avec gTTS ou Coqui TTS prévue.
- Vérification systématique du format et durée du fichier.

3. Problèmes de CORS et interopérabilité Web/API

- Le frontend tournant sur localhost:3000 et le backend sur localhost:8000, les navigateurs ont bloqué les requêtes `fetch()` à cause des politiques CORS.

Résolution :

- Intégration du middleware `CORSMiddleware` dans FastAPI pour autoriser l'origine localhost:3000.

4. Erreurs HTTP 422 et mauvaise gestion des fichiers

- L'API renvoyait des erreurs 422 Unprocessable Entity quand aucun fichier n'était envoyé ou que la clé `file` était incorrecte dans le `FormData`.

Solution :

- Ajout d'un contrôle côté JS (`if (!file)`) et validation serveur.
- Utilisation d'une clé explicite `"file"` côté client dans `formData.append()`.

5. Interception des requêtes par Internet Download Manager (IDM)

- Sur Windows, IDM interceptait les appels POST au lieu de laisser passer les fichiers audio, causant un faux téléchargement.

Solution :

- Désactivation temporaire d'IDM ou ajout de localhost à la liste d'exclusion dans les options IDM.

6. Absence de mémoire de conversation

- Le modèle répond à chaque requête sans tenir compte du contexte précédent.

Amélioration prévue :

- Ajout d'une session utilisateur ou d'une mémoire locale (`chat_history`) pour un dialogue cohérent à long terme.

8. Perspectives d'évolution

L'implémentation actuelle du chatbot médical vocal constitue une première étape vers une solution innovante d'assistance médicale pré-diagnostique. Plusieurs améliorations techniques et fonctionnelles sont envisagées à court et moyen terme. Cependant nous avons encore quelques difficultés avec l'envoi et la réception de la voie, nous avons néanmoins tester les modules indépendant avec des audio et ça marche.

8.1 Ajout de mémoire de contexte conversationnel

Actuellement, le chatbot répond à chaque requête de manière isolée. Une mémoire de session permettrait :

- De conserver l'historique des échanges pour un dialogue plus fluide et naturel.
- De poser des questions de suivi pertinentes.
- D'éviter les redondances inutiles.

Implémentation possible : stockage temporaire en base (ex : Redis), gestion de session utilisateur via ID ou token, ou intégration d'un wrapper mémoire (e.g. LangChain, RAG contextuel).

8.2 Version mobile Flutter

Pour une portabilité maximale, une application mobile Flutter est envisagée :

- Interface vocale interactive (microphone + affichage texte).
- Intégration directe avec le backend FastAPI.
- Mode hors-ligne partiel (STT/TTS local possible).
- Notifications, historique, et interface adaptée à des patients ou soignants.

Cela permettrait un usage en zone rurale ou sans PC, répondant à des besoins concrets du terrain.

8.3 WebSocket temps réel

Actuellement, les échanges voix → texte → voix sont séquentiels (requêtes HTTP POST classiques). L'usage de WebSocket permettrait :

- Une interaction bi-directionnelle temps réel,
- Une latence réduite,
- Un flux audio continu, idéal pour des conversations naturelles.

Cela offrirait une meilleure expérience utilisateur et faciliterait l'implémentation de conversations prolongées.

8.4 Multilingue / multivoix

L'architecture actuelle peut être étendue pour gérer plusieurs langues et voix :

- Reconnaissance vocale multilingue avec Whisper (plusieurs modèles disponibles).
- Traduction automatique (via MarianMT, NLLB, etc.).
- Synthèse vocale personnalisée selon le locuteur ou le dialecte (ElevenLabs, Coqui).
- Option d'interface en français, anglais, et langues africaines locales.

Cela rendrait le système inclusif et adapté à divers contextes culturels et géographiques.

7.5 Déploiement en milieu hospitalier

Le déploiement dans des structures hospitalières pilotes est un objectif à moyen terme :

- Intégration avec les dossiers médicaux électroniques (DME).
- Utilisation en salles d'attente, pour pré-triage automatisé.
- Formation des soignants à l'outil.
- Respect des normes de sécurité et confidentialité (ex : RGPD, HDS).

Des tests en conditions réelles permettront d'évaluer l'impact sur le temps de consultation, la satisfaction patient, et la charge mentale des professionnels.

Conclusion générale

Le développement de ce **chatbot médical vocal intelligent** constitue une avancée prometteuse dans l'accessibilité aux soins de santé, notamment dans les contextes à ressources limitées ou surchargés. En combinant **reconnaissance vocale (STT)**, **modèle de langage médical finement entraîné (LLM)** et **synthèse vocale (TTS)**, le système offre une interaction naturelle **voix-texte-voix**, simplifiant la communication avec une intelligence artificielle spécialisée.

L'architecture en **microservices avec FastAPI** permet une modularité, une maintenabilité, et une extensibilité remarquables. Grâce à un frontend léger basé sur HTML5 et JavaScript, même des utilisateurs sans compétences techniques peuvent tester et exploiter le service. Le choix de technologies open source (Whisper, Transformers, PEFT, Falcon, gTTS...) assure la reproductibilité et l'adaptabilité du projet.

Les tests menés jusqu'ici ont permis de valider les composants essentiels du pipeline, notamment la qualité de la transcription vocale et de la synthèse audio. Malgré l'absence temporaire du modèle final fine-tuné, la base fonctionnelle est solide. Des défis techniques comme la gestion des formats audio, les erreurs réseau ou les problèmes de CORS ont été identifiés et partiellement résolus, préparant le terrain pour une intégration fluide du modèle complet.

Enfin, plusieurs perspectives ont été proposées pour enrichir cette solution : **mémoire contextuelle, application mobile Flutter, support multilingue, WebSocket temps réel**, et **déploiement en milieu hospitalier**. Ce projet ouvre la voie à une nouvelle génération d'assistants médicaux intelligents, capables d'améliorer à la fois l'expérience patient et la charge des professionnels de santé.